

按一下按鈕，電腦怎麼知道？

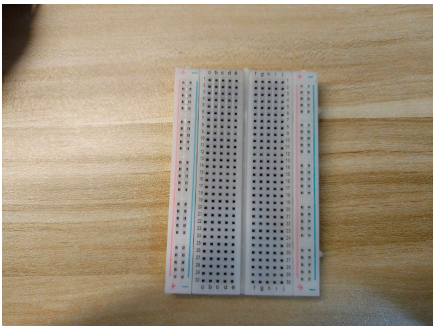
按鈕改變電路的連接，ESP32 讀取狀態，再把文字傳回電腦。

按鈕、電路與紀錄：

GPIO 是 ESP32 用來讀取或輸出訊號的接腳。本週用它讀按鈕，再把結果傳到電腦。

零件與功能

- | | | |
|---|----------|--------------------------|
| 1 | 一條線 | 用電表確認兩端是否相通。 |
| 2 | 麵包板 | 找出原本就在麵包板內接通的孔。 |
| 3 | 按鈕 | 比較放開和按下時，哪些腳接通。 |
| 4 | ESP32 程式 | 把 GPIO 讀值解釋成按下或放開，再送出紀錄。 |



先拿這些就好

電表、麵包板、杜邦線、四腳按鈕。

照片用來認實物；後面的孔位圖才用來找位置。

通斷測試時，不接電源。受測物不接 ESP32、USB、電池或其他電源。電表會用自己的電池進行小電流測試。

電表先回答：通，還是不通？

這次只用通斷功能，不量電流，也不量供電電壓。



先認三個位置

1. 黑筆插 COM。
在這台表的下方中間。
2. 紅筆插 VΩmA。
在下方右邊。左邊 10A 不用。
3. 旋鈕轉到通斷蜂鳴符號。
找聲波圖案，不是 200 Ω 的數字。實物標示不同時，先一起核對，不猜檔位。

實物照片的旋鈕在 OFF。
接好表筆後，還要選通斷檔。

先做自測，再測一條線

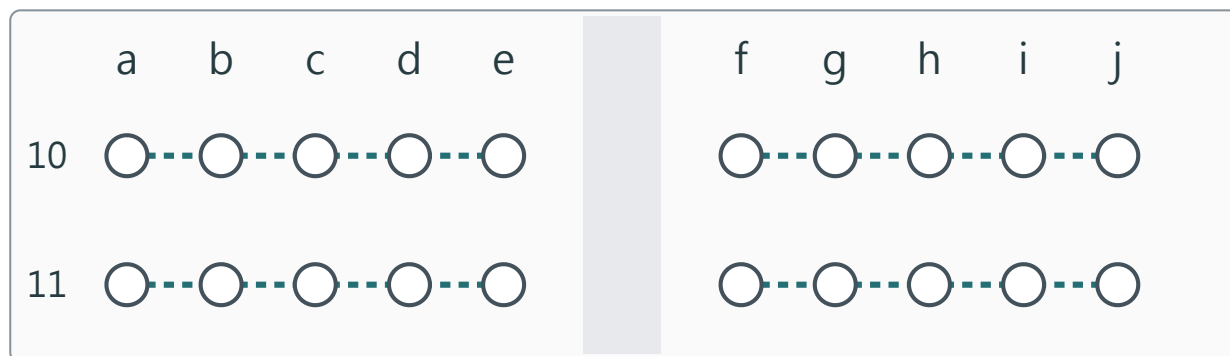
你的動作	預期	我的實際結果
兩個筆尖分開	不叫	_____
兩個金屬筆尖接觸	會叫	_____
同一條公對公線的兩端，各碰一筆	完好的線會叫	_____

怎麼判讀：電阻表示路徑對電流的阻礙。兩筆間的電阻夠小，通斷檔就會叫；例如完好導線。蜂鳴不一定表示故障。

結果不符時，檢查插孔、檔位與接觸。不要改接電源來試。通斷檔不能用在帶電電路。

麵包板：哪些孔原本就相通？

孔靠得很近，不一定相通。用電表驗證，不只看外觀。



中央槽把左右分開；虛線是麵包板內的金屬連接。

示意圖只畫中央接線區兩列，不含兩側電源軌；依實物字母與列號找孔。

先讀一個孔位

a10 是 a 欄、第 10 列。這張圖中，a10 到 e10 是同一組；f10 到 j10 是另一組。

用兩條杜邦線把測試位置引出來

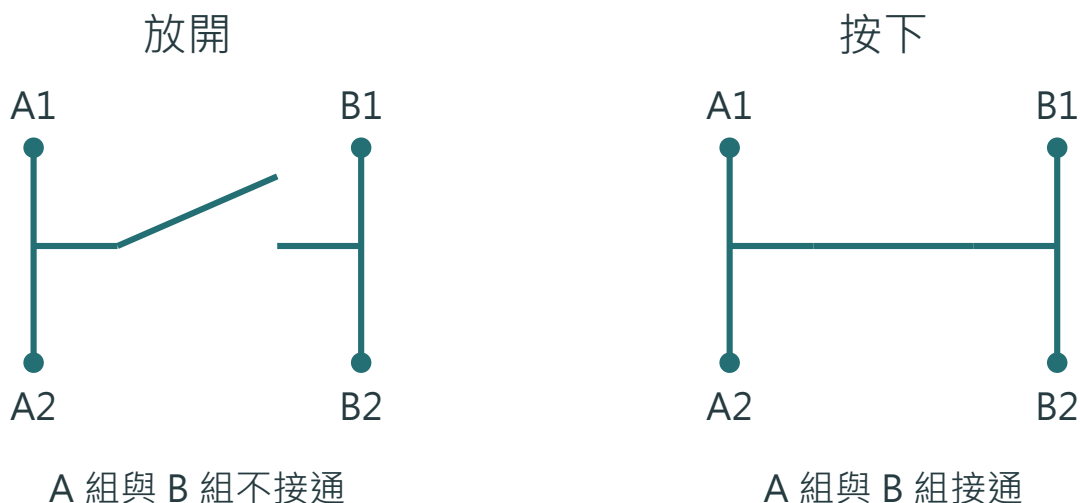
1. 在要比較的兩個孔，各插入一條公對公杜邦線。
2. 紅、黑表筆各碰一條線另一端的金屬針。
3. 兩條線末端不要互碰；不要把粗表筆硬塞進麵包板孔。

比較的兩孔	先猜：叫 / 不叫	實測：叫 / 不叫
a10 與 e10	_____	_____
a10 與 a11	_____	_____
a10 與 f10	_____	_____

核對：正常、未加其他接線的這類麵包板，三次應是「叫、不叫、不叫」。不符時先確認列號、線是否插到底，以及表筆接觸。

四腳按鈕，分成兩組接點

每組兩腳一直相通；按下時，兩組也會接通。



這是內部連接示意，不是腳位擺放圖。A1、A2、B1、B2 是本頁為說明測試而標的腳，必須實測辨認。

按鈕先不要裝進其他電路

1. 放開按鈕，用通斷檔比較各腳，找出原本就相通的兩腳，記為 A1、A2。
2. 確認另兩腳放開時也相通，記為 B1、B2。
3. 紅筆碰 A1、黑筆碰 B1。固定接觸後，再比較放開與按下。

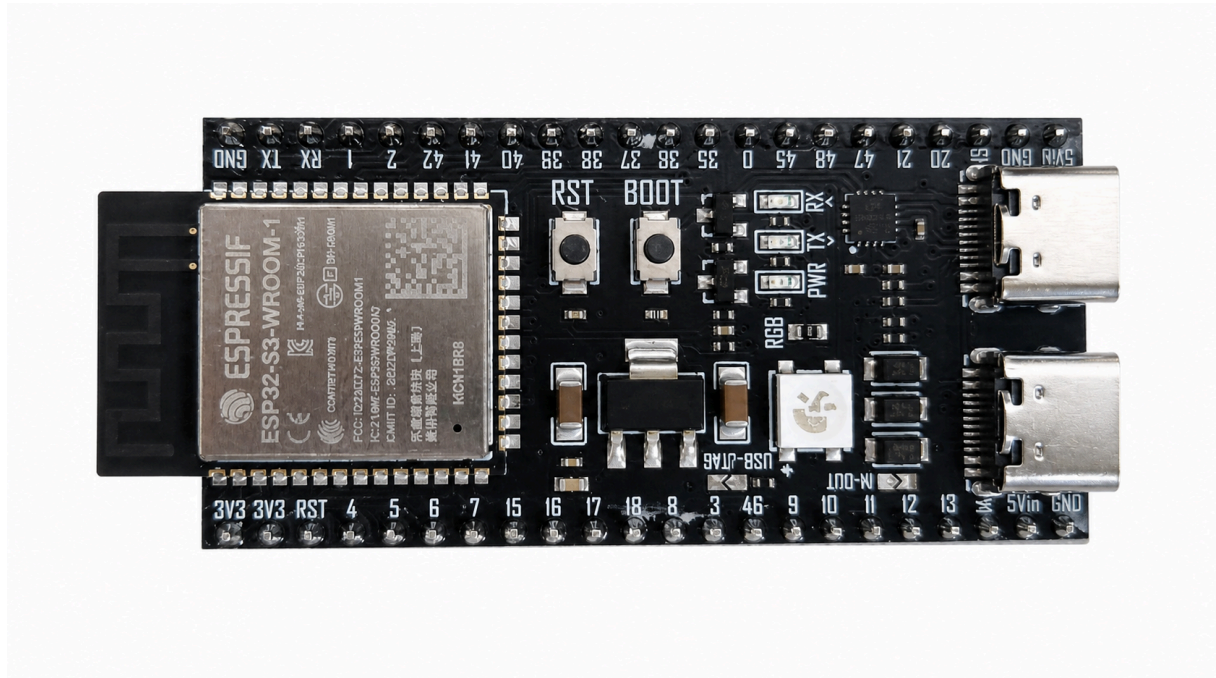
量哪兩腳	放開時的預期	按下時的預期
A1 與 A2 (同組)	會叫	會叫
A1 與 B1 (跨組)	不叫	會叫

最重要的判斷：要讓按鈕改變連通，必須從兩組各取一腳。只取同組兩腳，按不按都相通。

我的按鈕哪兩腳是同組？在紙上畫四個腳，把相通的一組圈起來。若測不出上表結果，先一起核對接觸與腳位，不接 ESP32。

拿出開發板，找到兩個腳位

先找印字，不接線。這一段只用 GPIO4 與 GND。



照片：課堂既有 YD-ESP32-S3 Type-A V1.5。把你的板子轉成相同方向：天線在左，兩個 USB 接頭在右。

GND 是共同的 0V 參考點。本週比較 GPIO4 和 GND 的電壓；V 是電壓單位「伏特」。

跟著找到，先用手指出來

這次找哪個腳	照片中的位置
印字「4」	下排，印字 RST 的右邊、5 的左邊。這是 GPIO4，不是「數第四個腳」。
印字「GND」	上排最左端，靠天線這一側。這次統一使用這個 GND。

板子先放桌上：放在乾燥、不導電的平面，不插進這塊 400 孔麵包板。稍後用公對母杜邦線連過去。

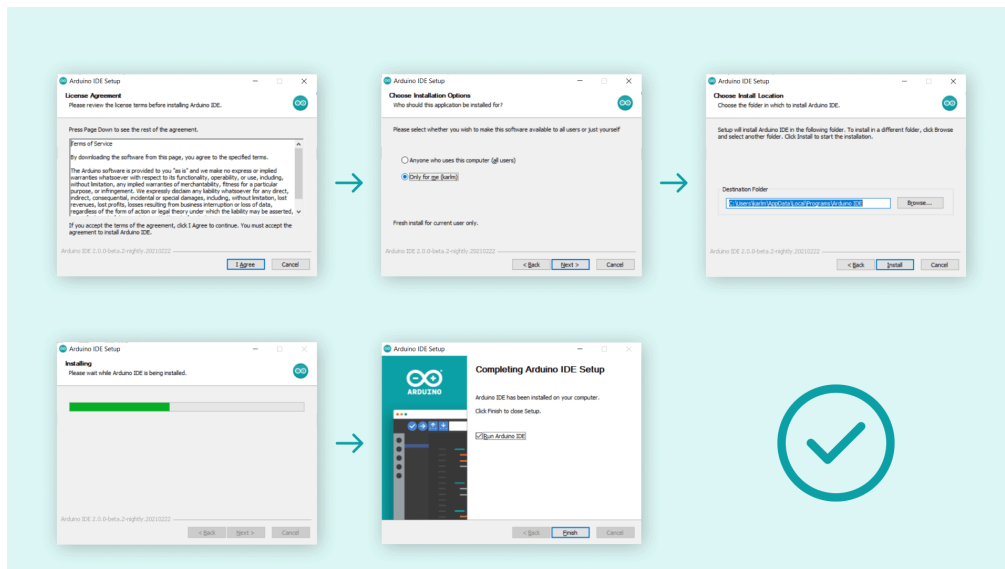
本接法只對應圖示板型。你的印字或板型不一樣，就停在辨認這一步，和全班一起核對，不靠外形猜腳位。板上若還有其他作品的接線，先斷開所有電源，另用空白器材準備本實驗。

先安裝 Arduino IDE

它是電腦上的軟體：我們用它寫程式，再把程式放進 ESP32。

以下以 **Windows 11 / Arduino IDE 2** 操作。板子先不用插 USB，也不接杜邦線。

1. 打開瀏覽器，進入 [Arduino 官方下載頁](#)。
2. 找 **Arduino IDE 2**，選 Windows 64-bit 的 **.exe 安裝檔**。不要選網頁版 Cloud Editor。
3. 下載完成後，開啟電腦的「下載」資料夾，連按兩下剛下載的 .exe。
4. 閱讀授權後，願意接受才按 **I Agree**。個人電腦可選 **Only for me**，再按 Next、Install。
5. 等安裝完成，勾選 **Run Arduino IDE**，按 Finish。已安裝 IDE 2 的同學直接從開始選單打開。



Arduino 官方安裝流程總覽，使用早期 IDE 2 畫面。實際版本號、使用者名稱與路徑不必相同；以目前安裝程式文字為準。

做完應看到：Arduino IDE 的程式編輯視窗。第一次開啟可能還在下載必要工具，等它完成。

學校電腦若沒有安裝權限，先一起處理，不繞過管理限制。本講義不要求雲端帳號，也不需要付費方案。

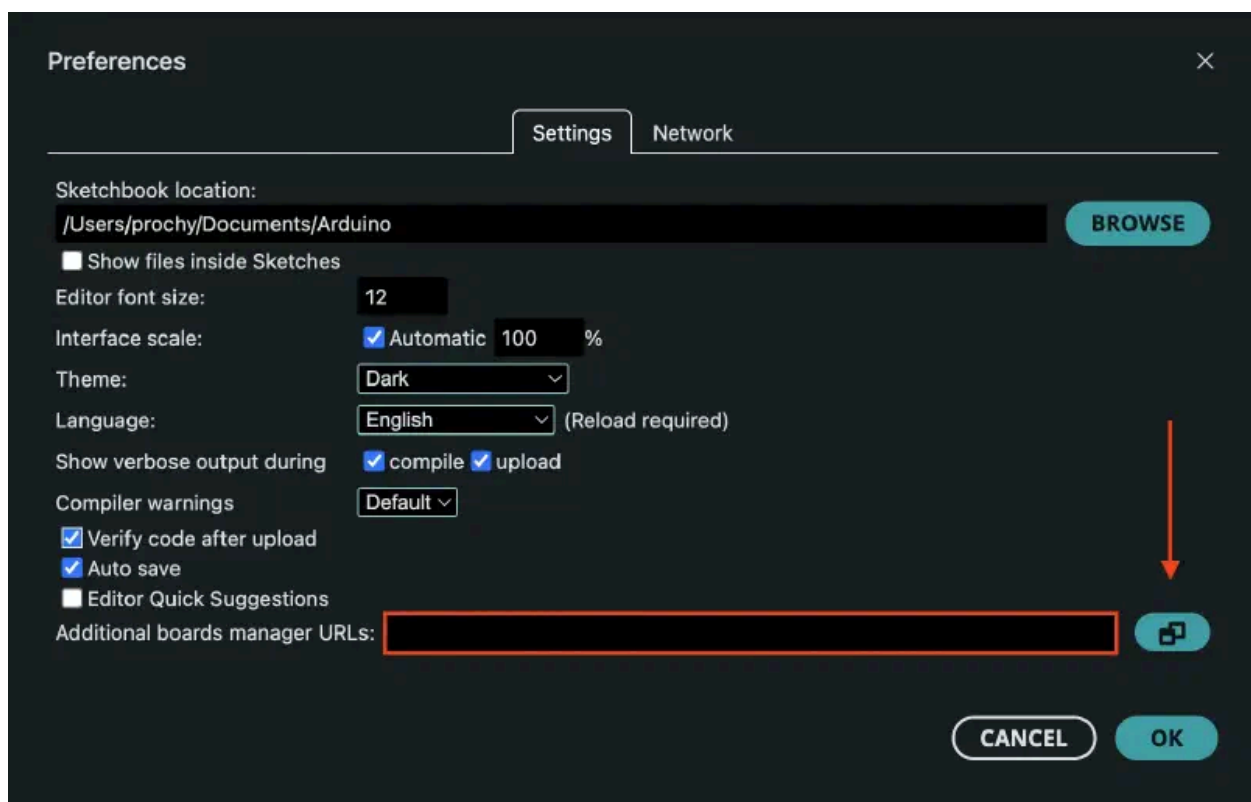
操作依據與畫面來源：[Arduino 安裝說明](#)。圖片未修改；版本差異見圖說。

告訴軟體去哪裡找 ESP32 支援

剛裝好的 Arduino IDE，還需要知道如何處理我們這種板子。

1. 在視窗上方點 **File (檔案) → Preferences (偏好設定)**。
2. 找到 **Additional boards manager URLs** 欄位。可以點右側按鈕開啟清單。
3. 貼上下面這個網址。若原本已有其他網址，保留它，另加一行；不要覆蓋別人的設定。

https://espressif.github.io/arduino-esp32/package_esp32_index.json



Espressif 官方偏好設定畫面。只找框出的 Additional boards manager URLs；不要照抄圖中個人路徑、主題或其他設定。

按 **OK** 關閉網址清單，再按偏好設定視窗的 **OK**。

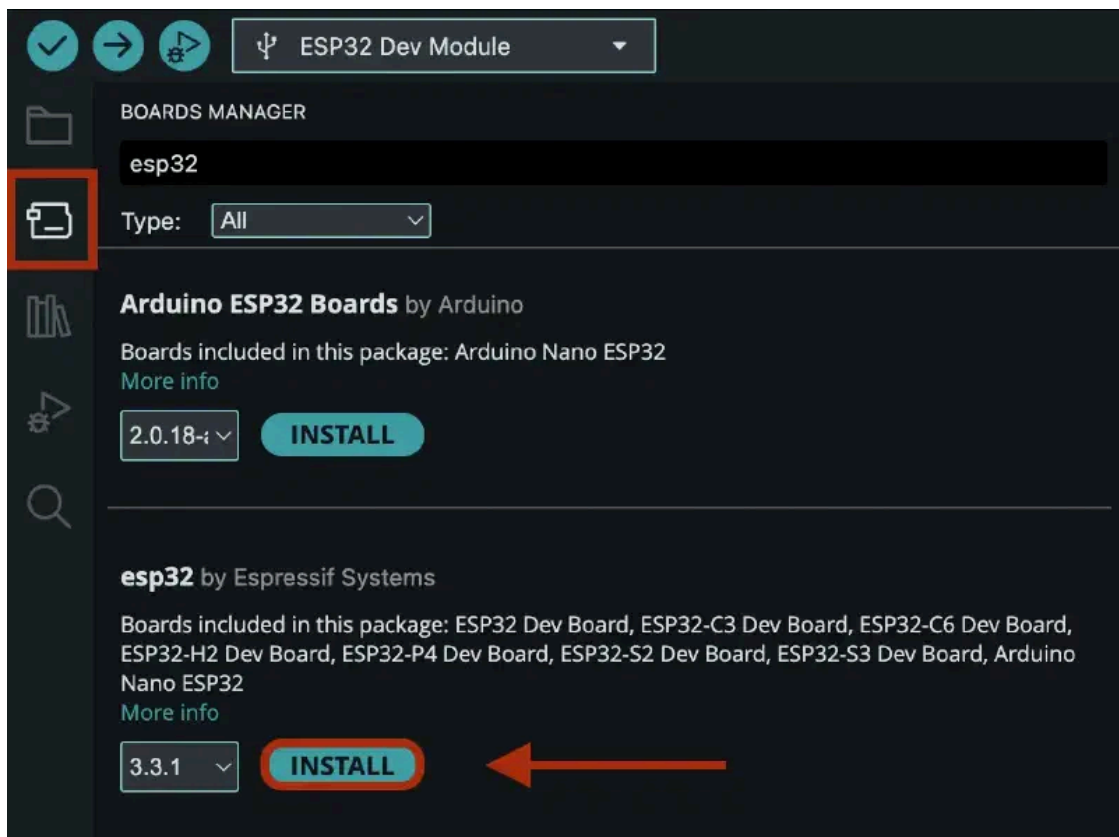
做完應確認：重新開啟 Preferences，剛貼上的網址還在。網址中是 `package_esp32_index.json`，沒有 `dev`。

操作依據與畫面來源：[Espressif 安裝教學](#)。圖片未修改；版本差異見圖說。

找到 esp32 by Espressif Systems

這套工具讓 Arduino IDE 能編譯程式，並上傳到 ESP32。

1. 點上方 **Tools (工具)** → **Board (開發板)** → **Boards Manager (開發板管理員)**。
2. 在搜尋欄輸入 **esp32**。找到作者是 **Espressif Systems** 的那一項。
3. 打開該項的版本選單，選 **3.3.11**，按 **INSTALL**，等下載與安裝完成。



Espressif 官方管理員畫面：本課要下面的 esp32 by Espressif Systems，不是上面的 Arduino ESP32 Boards。

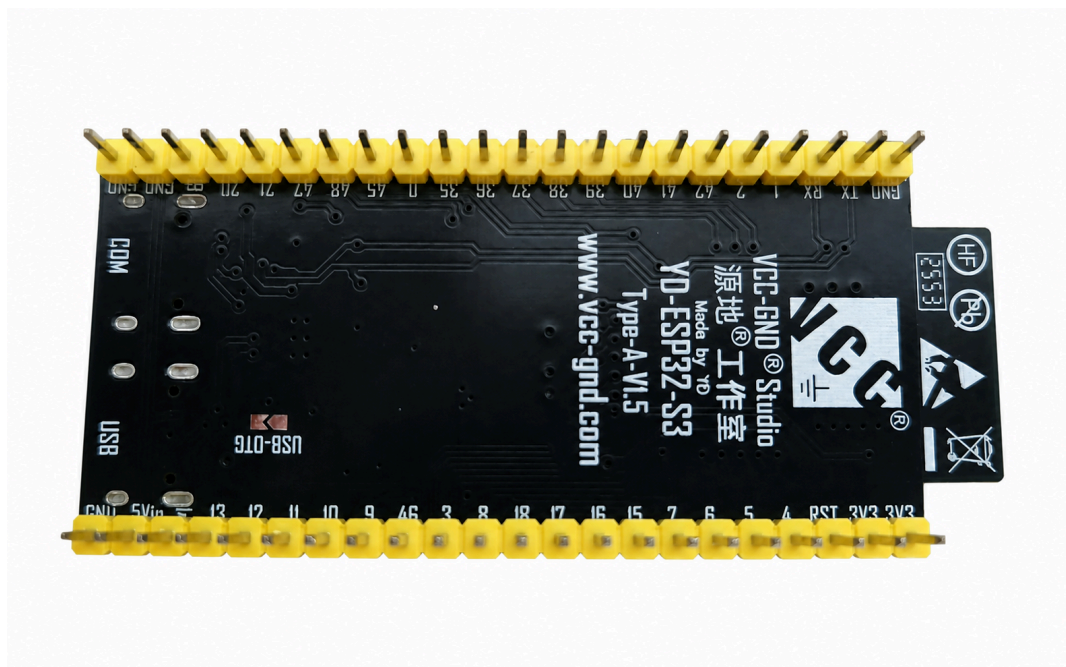
不要照抄圖片的版本與板名。圖中是舊版 3.3.1；本課選 3.3.11。上方 ESP32 Dev Module 也不是本課稍後要選的 ESP32S3 Dev Module。

做完應看到：這一項顯示 **3.3.11 installed** 或已安裝狀態。先確認不是仍在下載，再關閉並重新打開 Arduino IDE。

操作依據與畫面來源：[Espressif 開發板管理員教學](#)。圖片未修改；版本差異見圖說。

把 ESP32 接到電腦

使用可以傳資料的 USB 線；有些線只能充電，不能傳程式。



這是課堂板子的背面照片。本頁照片的 USB 接頭在左側；找印字 COM 對應的接頭，不靠正反面翻轉後的上下位置猜。

1. 板子沒有杜邦線、按鈕、電池或其他模組。把板子翻過來，找到印字 **COM** 的接頭。
2. 先看電腦軟體的 **Tools** → **Port** 清單。Port 是電腦用來識別連接裝置的通訊埠；現在沒有清單也沒關係。
3. 把 USB 線一端插電腦，另一端插板子的 **COM** 接頭。先只接這一塊板。
4. 把板子正面朝上放回乾燥、不導電的桌面，等 Windows 辨識，再重開 **Tools** → **Port**。

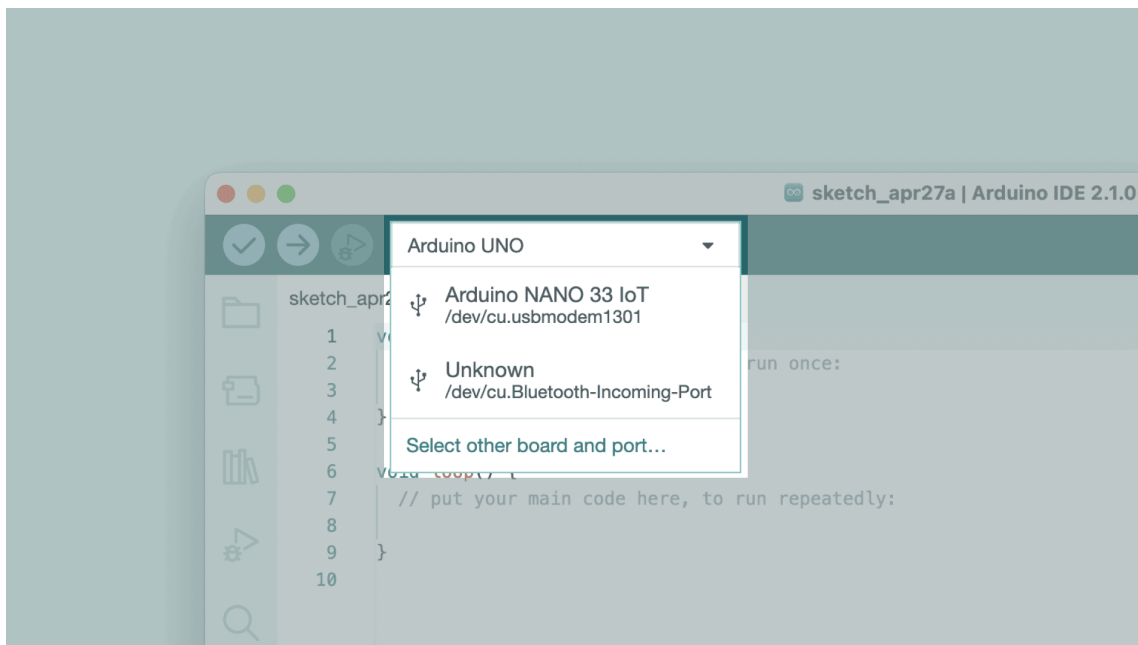
記下新增的名稱：COM_____。例如 COM8 只是某次電腦分配的號碼，不是全班都選 COM8。

板背的 **COM** 是接頭標示；電腦上的 **COM8** 之類是軟體名稱。PWR 燈亮只表示板上有供電，還不能證明資料線或程式傳輸正常。

若沒有新增 Port，先確認使用可傳資料的 USB 線及板上 COM 接頭；不要任選清單中的其他裝置。

選 ESP32S3 Dev Module

這個選單告訴軟體「程式要給哪一種板子」。另一個 Port 選單則指定連哪一塊。



Arduino 官方選單截圖，為 macOS / 其他板型示例。只看下方 Select other board and port 的入口；不選圖中的 UNO、Nano 或 Bluetooth。

1. 點頂端的開發板下拉選單，再點 **Select other board and port...**（選擇其他開發板與連接埠）。
2. 在 **Boards** 搜尋欄輸入 **ESP32S3 Dev Module**，點選完全相同的名稱。留意中間有 **S3**。
3. 在 **Ports** 清單，選第 9 頁剛記下的 COM 號碼，再按 **OK**。

你現在選的是	應該是什麼
Board (板型)	ESP32S3 Dev Module
Port (連接埠)	我的 COM_____，不是老師的固定號碼

清單若只寫 **Unknown**，不代表板子壞了；仍可手動選板型。若完全找不到 ESP32S3 Dev Module，回第 8 頁檢查安裝是否完成。

操作依據與畫面來源：[Arduino 板型與 Port 說明](#)；[圖見 Arduino 上傳教學](#)。圖片未修改；版本差異見圖說。

在 Tools 選單照表選一次

以下設定適用於圖示的 YD-ESP32-S3 N16R8 板卡。

先確認第 10 頁已選 **ESP32S3 Dev Module**，再點上方 **Tools (工具)**。每選完一項，重新打開 Tools 選下一項。

在 Tools 找這個名稱	選右邊這個值
Upload Speed	115200
USB Mode	Hardware CDC and JTAG
USB CDC On Boot	Disabled
Upload Mode	UART0 / Hardware CDC
Flash Mode	QIO 80MHz
Flash Size	16MB (128Mb)
Partition Scheme	16M Flash (3MB APP/9.9MB FATFS)
PSRAM	OPI PSRAM
Erase All Flash Before Sketch Upload	Disabled

Flash 存放程式，斷電仍保留；PSRAM 是執行時的額外暫存記憶體。其餘選項依表選，不必自行推算；本表適用圖示 N16R8 板與 esp32 3.3.11。

一起核對：重開 Tools，依序對照表格。找不到選項時，先核對板型和安裝版本，不隨便挑名字相近的項目。

這裡的 115200 是「把程式傳進板子」的速度。稍後訊息視窗也會用到 115200，但那是另一個設定，兩處都要各自選。

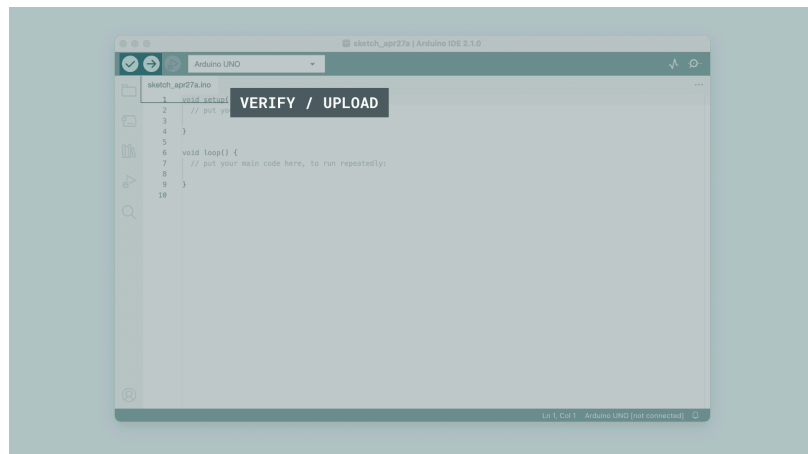
操作依據與畫面來源：[Espressif Tools 選單](#)；選項文字另核對本機 3.3.11。圖片未修改；版本差異見圖說。

把 Hello 程式放進 ESP32

「上傳」就是從電腦經 USB，把程式傳進板子；不是傳到網路。

1. 點 **File** → **New Sketch**（檔案 → 新增草稿），開一份新的程式，依第 10–11 頁核對板型、Port 與選項。
2. 在中間白色或黑色的程式編輯區點一下，按 **Ctrl+A** 全選，整段換成下面內容。
3. 按 **Ctrl+S**，命名為 **hello_first**。仍只接 USB，不接杜邦線。

```
void setup() {  
  Serial.begin(115200);  
}  
  
void loop() {  
  Serial.println("Hello");  
  delay(1000);  
}
```



Arduino 官方工具列截圖。左上方勾勾是 Verify；右箭頭是 Upload。圖中 UNO 不是本課板型，本課仍選 ESP32S3 Dev Module。

按左上方**右箭頭 Upload**，先等程式檢查，再等傳輸；中途不拔 USB。若跳出另存視窗，先完成儲存。

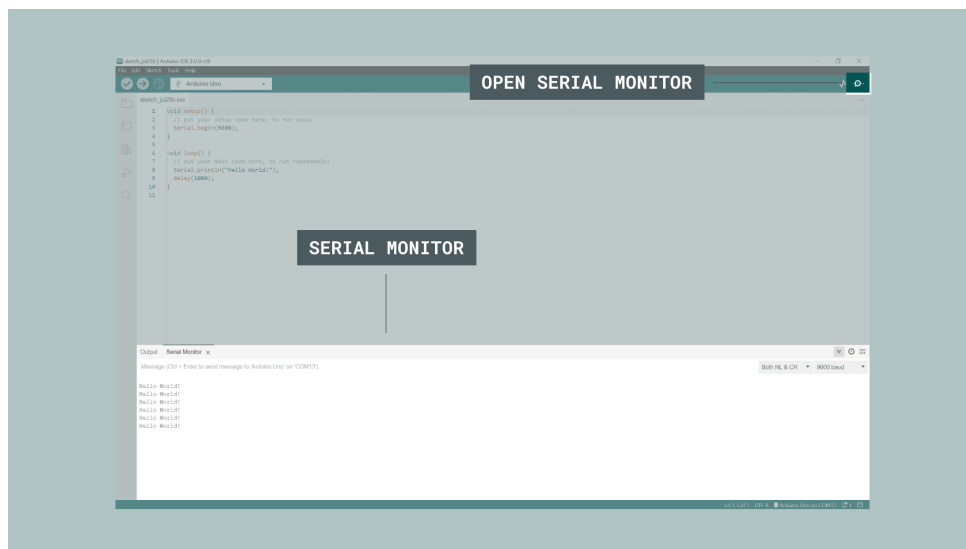
做完應看到：IDE 顯示 **Done uploading** 或上傳完成。只有「編譯成功」不等於已傳進板子。失敗時保留錯誤文字，確認板型、Port 與 USB 接頭。

操作依據與畫面來源：[Arduino 上傳教學](#)。圖片未修改；版本差異見圖說。

打開序列監控視窗，看 Hello

Serial Monitor (序列監控視窗) 就是顯示板子傳回文字的地方。

1. USB 保持插著。點 **Tools** → **Serial Monitor** (工具 → 序列監控視窗) ；或點右上方對應圖示。
2. 在下方訊息區找到 **baud** 速度選單，選 **115200**。不要照圖片選 9600。
3. 看下方的 **Serial Monitor** 分頁，不是旁邊顯示編譯訊息的 **Output** 分頁。



Arduino 官方示例只用來辨認右上入口與下方訊息區。本課是 ESP32S3、115200、文字 Hello；不是圖中的 UNO / 9600 / Hello World。

預期每隔約一秒多一行

```
Hello  
Hello  
Hello
```

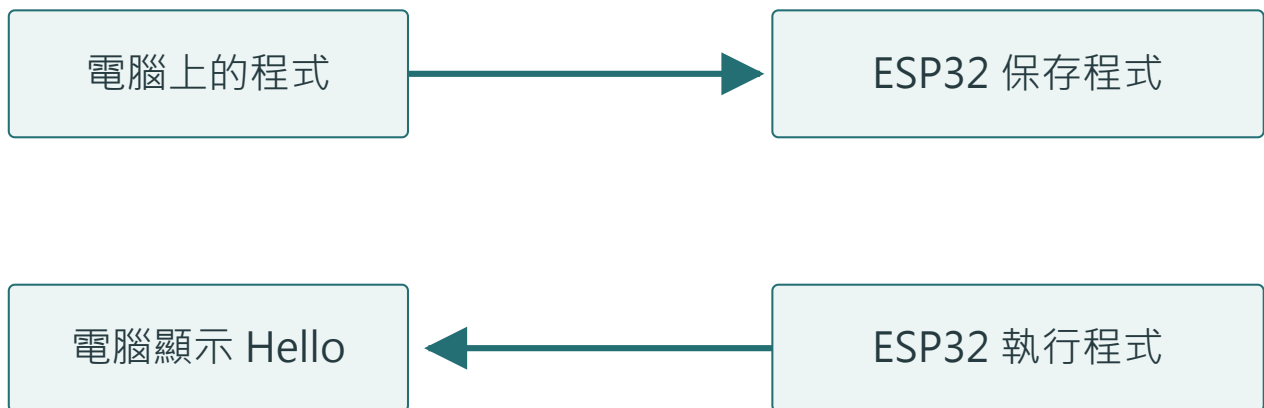
這是程式的預期示例，不是本次實測紀錄。訊息區應持續新增 Hello；開發板仍只接 USB。

再試一次：關閉 Serial Monitor 分頁，把程式的 "Hello" 改成 "Hi"，儲存並再按 Upload。重新開啟訊息視窗後，應改成 Hi。

操作依據與畫面來源：[Arduino Serial Monitor 教學](#)。圖片未修改；版本差異見圖說。

剛才 Hello 是誰送出來的？

上傳傳的是程式；Serial 傳回的是執行結果。



上排：按 Upload 後，程式經 USB 傳進板子。下排：板子執行後，把文字經 USB 傳回電腦。這是資訊流，不是電源接線圖。

剛才看到的程式	先用這句話理解
<code>setup()</code>	板子開始執行時，先做一次準備。
<code>Serial.begin(115200)</code>	準備傳文字，速度用 115200；訊息視窗也要選相同值。
<code>loop()</code>	裡面的動作會反覆做。
<code>Serial.println("Hello")</code>	送出 Hello，然後換行。
<code>delay(1000)</code>	等約 1000 毫秒，也就是 1 秒。

先猜，再看：把現在的 Hi 改成 Bye，只儲存、不按 Upload。訊息區會繼續顯示 Hi，還是變成 Bye？觀察後說明理由。

這個小實驗沒有使用 Wi-Fi，也沒有連雲端。Arduino IDE 是電腦上的工具；真正重複送出文字的是 ESP32。

先換程式，再拔 USB 接按鈕

剛才只會傳回 Hello；現在換成能回報「按下 / 放開」的程式。

在剛才的 Arduino IDE 點 **File → New Sketch**，將編輯區全選，換成下面整段程式，再用 Ctrl+S 存成 **button_follow_along**。板子仍只接 USB，沒有杜邦線。

開新草稿，整段貼上並上傳

```
const int BUTTON_PIN = 4;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

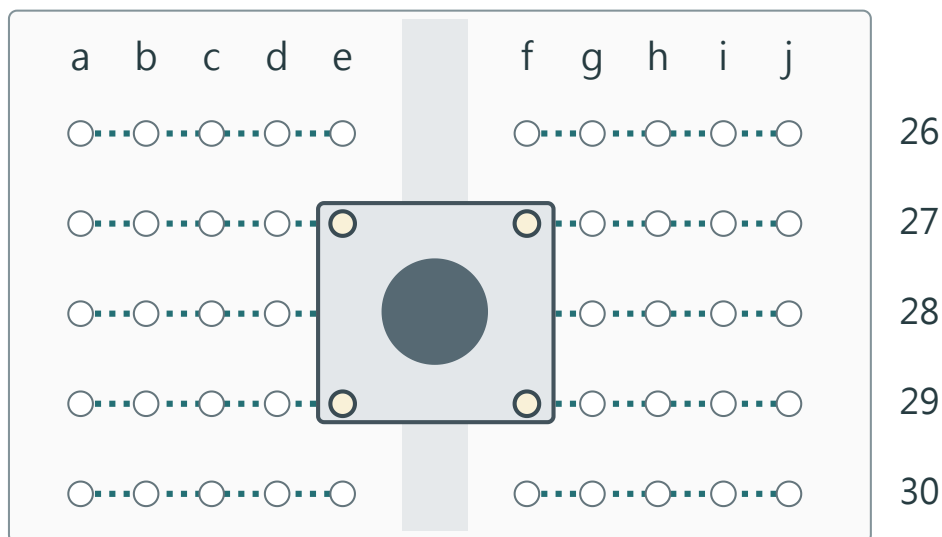
void loop() {
  int value = digitalRead(BUTTON_PIN);
  if (value == LOW) {
    Serial.println("pressed");
  } else {
    Serial.println("released");
  }
  delay(300);
}
```

1. 依第 10–11 頁核對板型、Port 與選項；新視窗不一定沿用前一份設定。先關閉 Hello 草稿的訊息視窗。
2. 按左上方右箭頭 **Upload**。完成後點 **Tools → Serial Monitor**，選 115200。
3. 現在還沒有接按鈕，應持續出現 **released**，意思是「放開」。這是預期畫面，不是本次實測紀錄。
4. 看到後，關閉訊息分頁、拔掉 USB，確認 PWR 燈熄滅。

未完成上傳或看不到 released，先核對程式、Port 與 USB 接頭，不先接線猜原因。

先放按鈕，暫時不接開發板

拿剛才已測出兩組腳的按鈕，跨中央槽放入。



按鈕四腳：e27、f27、e29、f29

俯視孔位圖。按鈕跨過中央槽；圓點標出四個腳插入的位置。圖中沒有開發板接線。

1. 找到麵包板第 27 列與第 29 列。
2. 將四腳放在 **e27**、**f27**、**e29**、**f29**。若腳距不能自然對孔，不硬壓。
3. 方向要符合：第 27 列兩腳原本同組；第 29 列兩腳是另一組。

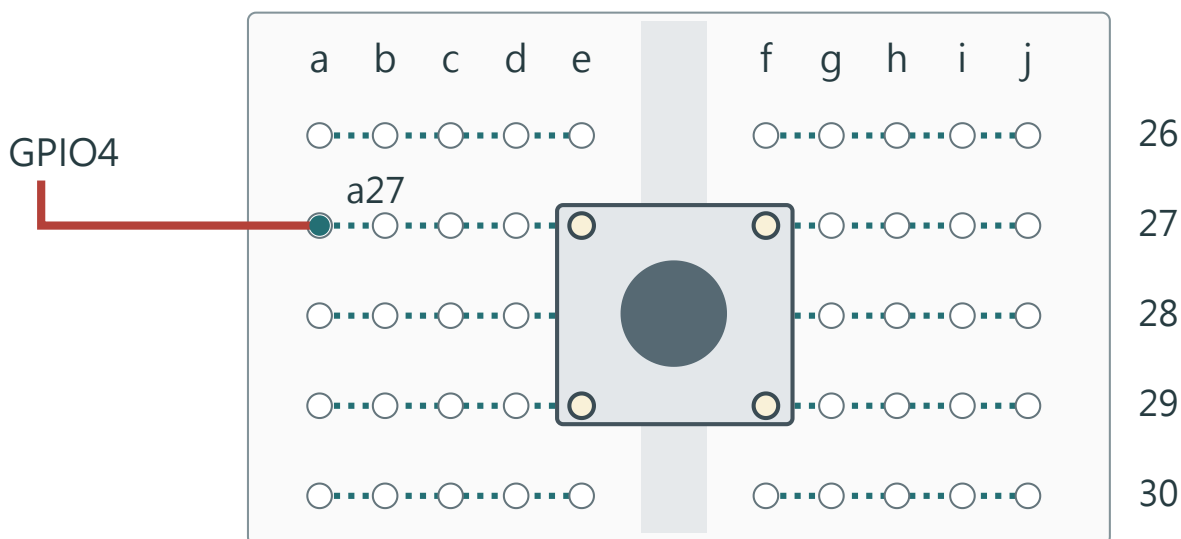
用剛才學會的通斷測試一起核對

測哪裡	預期
a27 與 j27，按鈕放開	會叫：跨槽兩腳是同組
a29 與 j29，按鈕放開	會叫：另一組也相通
a27 與 a29，先放開，再按住	不叫 → 會叫

沿用兩條公對公線引出測點。結果不同，先找按鈕方向、孔位與接觸；此時不接 USB。完成後移除量測延長線，電表轉 OFF、表筆放旁邊。

GPIO4 接到 a27

拿一條公對母杜邦線。線色只是方便辨認，接點才是重點。



按鈕四腳：e27、f27、e29、f29

接線關係圖，不是開發板實體排針排列圖。GPIO4 的實物位置請對照第 5 頁。

這條線只有兩端

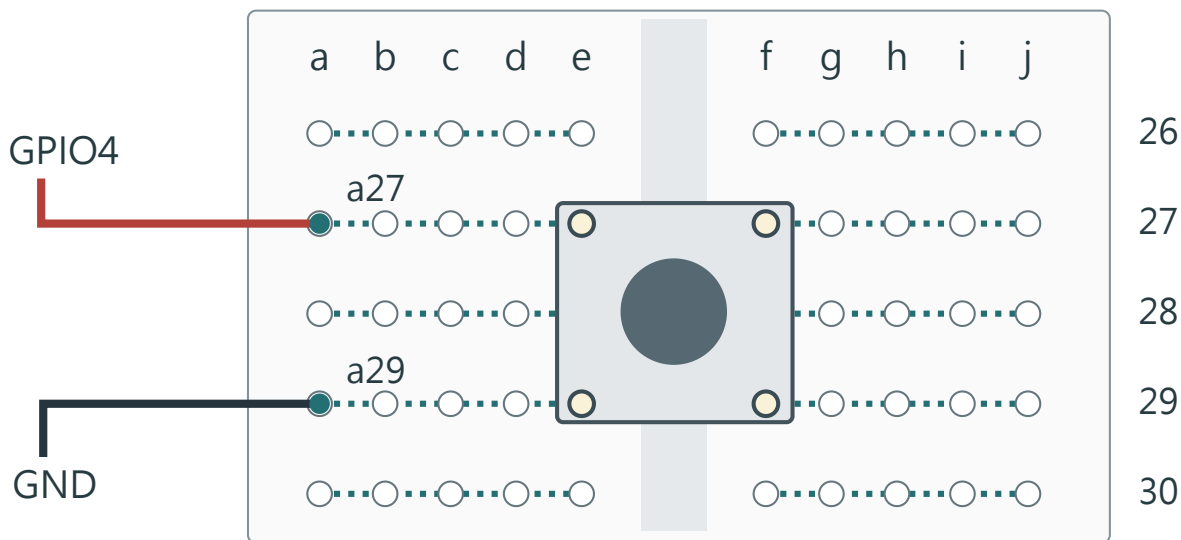
哪一端	接哪裡
母頭：有插孔的一端	套到開發板印字「4」的排針
公頭：有金屬針的一端	插麵包板 a27

按鈕一腳已在 e27；a27 和 e27 是同一組孔，所以這條線已連到按鈕的一組腳。

一起看目前狀態：板子放桌上，只有這一條線；USB 沒插，3V3、5Vin、GPIO5 都沒有接線。

GND 接到 a29

拿第二條公對母杜邦線。第一條線保持不動。



按鈕四腳：e27、f27、e29、f29

目前完整接法：GPIO4 → a27；GND → a29。兩線分別連到按鈕的兩組，不是接在同一組。

哪一端	接哪裡
母頭	套到第 5 頁指定的 GND 排針
公頭	插麵包板 a29

全班一起照圖核對，現在仍不插 USB

1. 按鈕仍在 e27、f27、e29、f29，方向沒有改變。
2. GPIO4 的線在 a27；GND 的線在 a29。不是 a28，也不是都在第 27 列。
3. 只有這兩條開發板接線；沒有線接 3V3 或 5Vin。金屬針沒有意外碰到旁邊的腳。
4. 電表已 OFF，表筆、量測延長線與其他模組都已移開。

這一段不接 GPIO5，也不用另外接一顆電阻。接點或板型還不確定，就保持斷電，一起對照處理；不靠通電試錯。

按下、放開，看文字改變

現在才插回板背標示 COM 的 USB 接頭，不用重新上傳。

1. 打開 Serial Monitor，選同一塊板子的 Port 與 **115200**。
2. 先不碰按鈕，看見什麼？接著按住約一秒，再放開約一秒。
3. 一起重複三次，對照下表。不要快速點一下就放開。

手的動作	預期會持續出現	我看到的文字
放開	released	_____
按住	pressed	_____
再次放開	released	_____

預期畫面示例，不是實測紀錄

```
released
released
pressed
pressed
released
```

這支程式約每 0.3 秒讀一次並印一行。按住時出現多行 `pressed` 是這次程式的預期行為，不是「按了很多次」，也不能用來判定按鈕彈跳。

文字不變：先拔 USB，再回第 16–18 頁核對方向與接點。完全沒有文字：先核對 Port、接頭與 115200。改線時不插 USB。

按下會接通，為什麼讀到 LOW ？

這個接法：放開是 HIGH，按下才是 LOW。

GPIO4 比較接點和 GND 的電壓。GND 當作 0V；接點接近 3.3V 時讀到 HIGH（高），接近 0V 時讀到 LOW（低）。電流是電荷流動；這裡的 HIGH / LOW 不是電流大小。

電阻在哪裡？在開發板上的 **ESP32 晶片內部**，不是按鈕或麵包板裡。第 15 頁的 **INPUT_PULLUP** 會開啟這顆「上拉電阻」，把 GPIO4 接向 3.3V。

放開：GPIO4 約 3.3V → HIGH



按鈕斷開，這條支路幾乎沒有電流；不是電流最強。

按下：GPIO4 約 0V → LOW



電流路徑：3.3V → 上拉電阻 → 按鈕 → GND。

同一個電路的兩種狀態。框內是晶片內部，框外是已接好的線與按鈕。電壓為正常接線的近似說明，不是本次實測值。

放開：按鈕切斷通往 GND 的路。GPIO 輸入只取用極小電流，電阻兩端幾乎沒有電壓差，所以 GPIO4 仍接近 3.3V。

按下：GPIO4 經按鈕連到 GND，接點就接近 0V。此時有小電流經過上拉電阻與按鈕，但「有電流」不等於 HIGH。

這顆電阻會限制按下時的電流；不能改成把 3V3 電源腳直接用線接到 GND。

HIGH / LOW 是電壓的高低，不是電流的強弱。

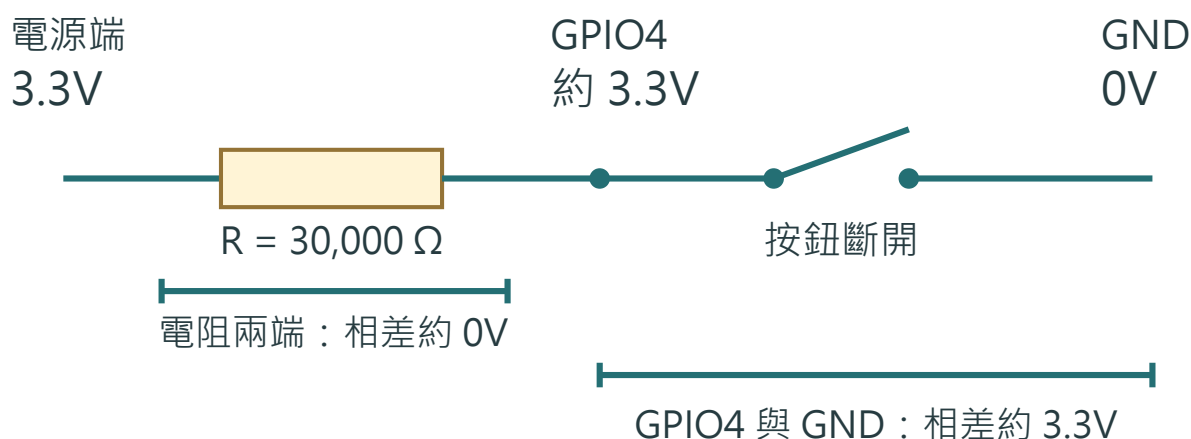
沒按按鈕，為什麼是 3.3V？

3.3V 電源與 GPIO4 中間連著一顆電阻，位於 ESP32 晶片內。

沒按時，通往 GND 的路斷開。流過電阻的電流幾乎為零，電阻兩端幾乎沒有電壓差。因此 GPIO4 與電源端一樣，約為 3.3V，讀到 HIGH。

歐姆定律 $V = I \times R$ ：這裡 V 是電阻兩端的電壓差（伏特 V）， I 是流過電阻的電流（安培 A）， R 是電阻值（歐姆 Ω ）。

為什麼要分兩個電壓？電壓要比較兩個位置：電阻的電壓差比「左端與右端」；GPIO 的電壓比「GPIO4 與 GND」。比較的位置不同，數值可以不同。



為了算數，假設晶片內上拉電阻 $R = 30,000 \Omega$ ；不是實物精確阻值，也不是要外接這顆電阻。下方短線標示比較的位置，不是新增接線。

按鈕斷開，用三步算出 HIGH

① 按鈕斷開： $I \approx 0 \text{ A}$

② 電阻兩端電壓差 $\approx 0 \times 30,000 = 0\text{V}$

③ GPIO4 對 GND $\approx 3.3 - 0 = 3.3\text{V} \rightarrow \text{HIGH}$

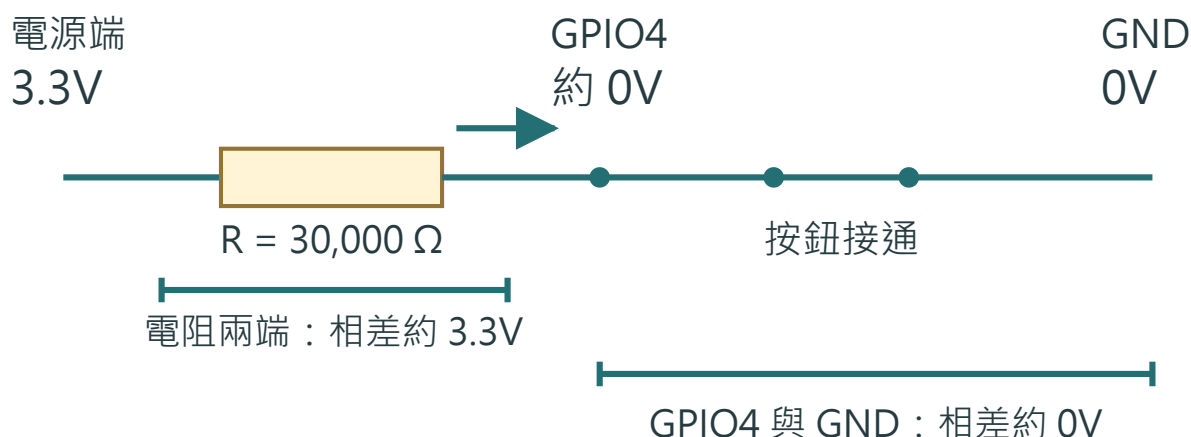
$\approx$ 表示「約等於」。忽略 GPIO 的微小輸入電流，只看狀態穩定後的近似結果。

有電阻，不代表一定會降低電壓。流過的電流幾乎是零，電阻兩端的電壓差也就幾乎是零。

按下後，為什麼變成 0V ？

GPIO4 與 GND 之間只有接通的按鈕和導線。

按下時，GPIO4 接到 GND (0V)。雖然仍連著 3.3V，但那一側隔著大電阻；GND 這一側的電阻幾乎為零。因此 GPIO4 接近 0V，讀到 LOW。



沿用上一頁的假設 $R = 30,000 \Omega$ 。電阻左端約 3.3V、右端約 0V；GPIO4 與 GND 則都約 0V。箭頭表示電流方向，不是另一條線。

3.3V 沒消失，電壓差在電阻兩端

① GPIO4 對 GND 約 0V → LOW

② 電阻兩端電壓差 $\approx 3.3 - 0 = 3.3V$

③ $I = V \div R \approx 3.3 \div 30,000 = 0.00011 A$

電源 → 上拉電阻 → 接通的按鈕 → GND，形成電流路徑；忽略導線與按鈕的微小電阻。

$0.00011 A = 0.11 mA$ ，因為 $1 A = 1,000 mA$ 。這是限流後的小電流，不是「沒有電流」。

記住：HIGH / LOW 看 GPIO 對 GND 的電壓，不看電流強弱。以上是計算示例，不是實測；不拆電阻、不改接 3V3，也不切電表到電流檔。

如果把電阻換成導線？

假設 3.3V 電源直接連到 GPIO4，中間不再有上拉電阻。

只看圖理解，不要照圖接線。這不是操作練習，也不是拆除晶片內電阻的步驟。按下圖中的按鈕會使電源短路，可能損壞板子。

① 沒按：GPIO4 仍約為 3.3V



電源正常供應 3.3V，按鈕處斷開，沒有形成電源到 GND 的短路路徑。

GPIO4 直接連著電源，所以 GPIO4 與 GND 的電壓差約為 3.3V。導線兩端的電壓差則幾乎是零。

② 按下：電源直接接到 GND，形成短路



電源 → 導線 → 接通的按鈕 → GND。失去上拉電阻的限流，這不是正常的 LOW 操作。

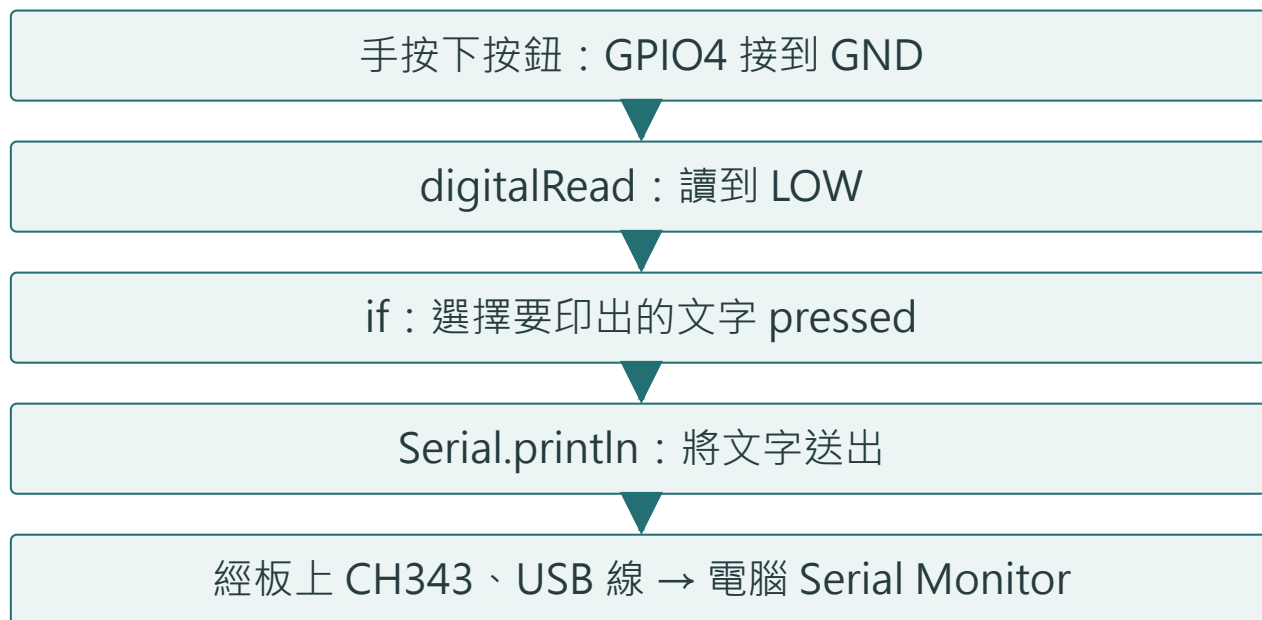
短路可能造成很大的電流、電源電壓下降或保護動作。GPIO4 與 GND 的電壓差要看電源和線路，不能保證固定值。

不能同時假設「電源維持正常 3.3V」和「整條導線都是 0V」。實際導線、按鈕和電源都有電阻或供電限制。

有上拉電阻：按下時限制電流，GPIO4 可接近 0V，而電源仍正常供應 3.3V。
換成導線：按下時把電源短接到 GND，不能再當成正常電路判讀。

LOW 怎麼變成電腦上的 pressed ？

程式依照 GPIO4 的讀值，選擇傳回 pressed 或 released。



按下時的資訊流。箭頭表示處理與傳送順序，不是電流路徑，也不是要新增的杜邦線。

圖中的 UART 是 ESP32 傳送文字的通訊介面；CH343 把 UART 訊號轉接到 USB，讓電腦收到文字。

文字是程式依讀值選出來的

第 15 頁的 `digitalRead(BUTTON_PIN)` 先讀 GPIO4，存進 `value`（暫存讀值的名字）。
`if (value == LOW)` 再問：「讀值是不是 LOW？」是就印 `pressed`；不是就印 `released`。

這個接法的動作	GPIO4 讀值	程式送出的文字
放開	HIGH	released (放開)
按下	LOW	pressed (按下)

想一想：若只把程式中的 "pressed" 改成 "Hello"，按下時 GPIO4 會變成 HIGH 嗎？
不會。接線沒變，仍是 LOW；改變的只有送到電腦的文字。

結束操作時，關閉 Monitor、拔掉 USB。

原理核對：[Fluke 通斷測試](#)；[Espressif GPIO](#)。預期結果為教學推演，非本次實測。

兩個按鈕，讓數字加減

加鍵讓數字增加，減鍵讓數字減少；結果顯示在 Serial Monitor。



資訊流，不是接線圖。數字由 ESP32 計算，經 USB 傳回電腦。

操作	預期反應
按加鍵	畫面上的數字增加。
按減鍵	畫面上的數字減少。
兩鍵都放開	畫面不再新增數字。

已有一顆按鈕、兩條公對母線與麵包板。再拿一顆按鈕、一條公對母線、一條公對公線。本作品不需要 OLED、Wi-Fi 或手機。

數字從 0 開始。這個程式只做加減，沒有名額、滿額提示或其他操作規則。

先讓板子準備好讀兩顆按鈕

剛才只用 GPIO4；這次增加 GPIO5，兩個腳都用來讀按鈕。

1. 關閉 Serial Monitor，拔 USB，移除其他電源。PWR 熄滅後，把開發板上的杜邦線全部拔下；按鈕可留在麵包板上。
2. 開發板單獨放桌上，只接板背標示 **COM** 的 USB 接頭。
3. 有教師的 IoT 資料夾就沿用；否則開課程 [GitHub](#)，按 **Code** → **Download ZIP**，下載後右鍵「全部解壓縮」。
4. 在資料夾依序開 **IOT_Introduction** → **docs** → **teaching_drafts** → **week2_redesign** → **counter_practice**。IDE 選 **檔案** → **開啟**，開裡面的 **counter_practice.ino**，不用抄第 30 頁。
5. 選 **檔案** → **另存新檔**，存到自己的練習位置，名稱仍用 **counter_practice**。依第 10–11 頁選板型、Port 與選項，按 **Upload** 等上傳完成。
6. 開 **Tools** → **Serial Monitor**，選 **115200**。不按 BOOT，短按一次板上 **RST**，應看到下面這一行。

0

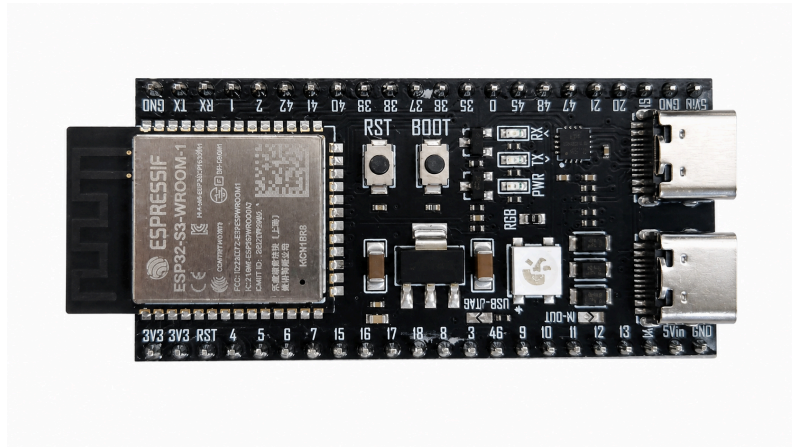
畫面上的 **0** 是計數器的起始數字。

兩鍵都放開時，不會持續新增數字。看不到 0，確認 COM 接頭、自己的 Port 與 115200，再短按 RST；不要更動接線嘗試解決上傳問題。

看到起始文字後，關閉 Monitor、拔 USB，確認 PWR 熄滅。程式已將 GPIO4、GPIO5 設為讀取按鈕的輸入腳；接線時保持斷電。

原本的加鍵留下，再放一顆減鍵

安裝與測試按鈕時，不接開發板，也不插 USB。



新增加的 GPIO5 是下排印字「5」，在「4」與「6」之間。不是從旁邊數第五腳。圖示板型不同時，先停止並核對。

哪顆按鈕	四腳放在哪裡	稍後接哪個 GPIO
原本那顆，貼「+」	e27、f27、e29、f29	GPIO4
新增加的，貼「-」	e21、f21、e23、f23	GPIO5

減鍵也要測，不用外形猜方向

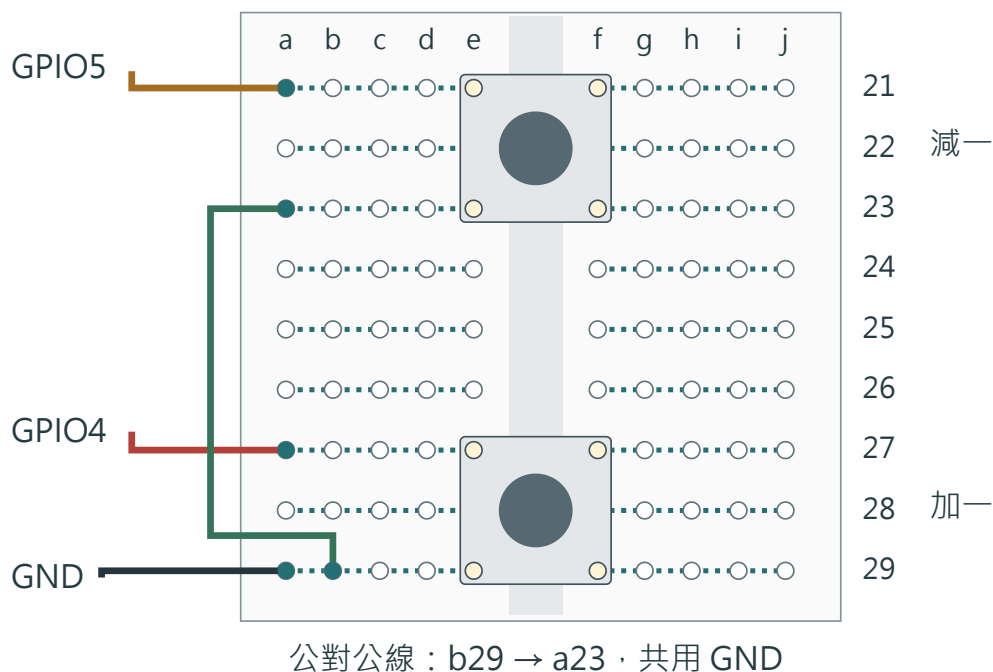
沿用第 2–4 頁的電表通斷檔，以及公對公線引出測點。兩顆按鈕現在都沒有接開發板。

減鍵測點	預期
a21 與 j21，放開	會叫，同一組腳
a23 與 j23，放開	會叫，另一組腳
a21 與 a23，放開 → 按住	不叫 → 會叫

測試不符就停在方向與接觸檢查，不通電試錯。完成後移除量測線，電表 OFF、表筆放旁邊。

四條線，連好加鍵與減鍵

USB 仍拔除。兩顆按鈕共用同一個 GND。



俯視關係圖，圓點表示線插入的孔；線段交叉但沒有圓點，不表示相接。板卡放桌上，不插在麵包板；以孔位與接點辨認，不靠線色。

順序	線材	兩端接點
1. 恢復加鍵	公對母	GPIO4 → a27
2. 恢復 GND	公對母	第 5 頁指定 GND → a29
3. 增加減鍵	公對母	GPIO5 → a21
4. 讓減鍵也接到 GND	公對公	b29 → a23

為什麼用 b29？a29 已插 GND 線，b29 和 a29 是麵包板內相通的孔。從 b29 拉線到 a23，就是把 GND 接到第二顆按鈕的另一組腳。

核對四條線：不接 3V3、5Vin 或其他模組。GPIO4 與 GPIO5 不可接在同一列相通的孔。確認接線正確，移除電表與量測線後，才接 USB。

加鍵增加，減鍵減少

看 Serial Monitor 中最後新增的數字。

1. 插回 **COM USB**，開 Serial Monitor，選 **115200**。
2. 兩鍵放開，短按 RST，畫面應出現 **0**。
3. 只按加鍵，看到數字往上增加後放開。
4. 只按減鍵，看到數字往下減少後放開。

預期畫面示例

```
0
1
2
3
2
1
```

這段示例是三次加一、接著兩次減一的結果。**不是要求按三下就必定到 3**；手指按住多久、程式讀到幾次，會影響增加或減少的次數。

畫面只印數字。每新增一行，表示 ESP32 剛做了一次加一或減一；放開後沒有新數字是正常的。

接線仍使用第 28 頁的四條線。需要改線時，先拔 USB。

雙按鈕計數器完整程式

counter_practice.ino：讀按鈕、加減數字、顯示結果。

```
// counter_practice：加鍵 GPIO4、減鍵 GPIO5，按鈕另一端接 GND。
// 適用已核對的 YD-ESP32-S3 Type-A V1.5；改線前拔 USB。
const int PLUS_PIN = 4;
const int MINUS_PIN = 5;
int count = 0;                                // 記住目前數字。

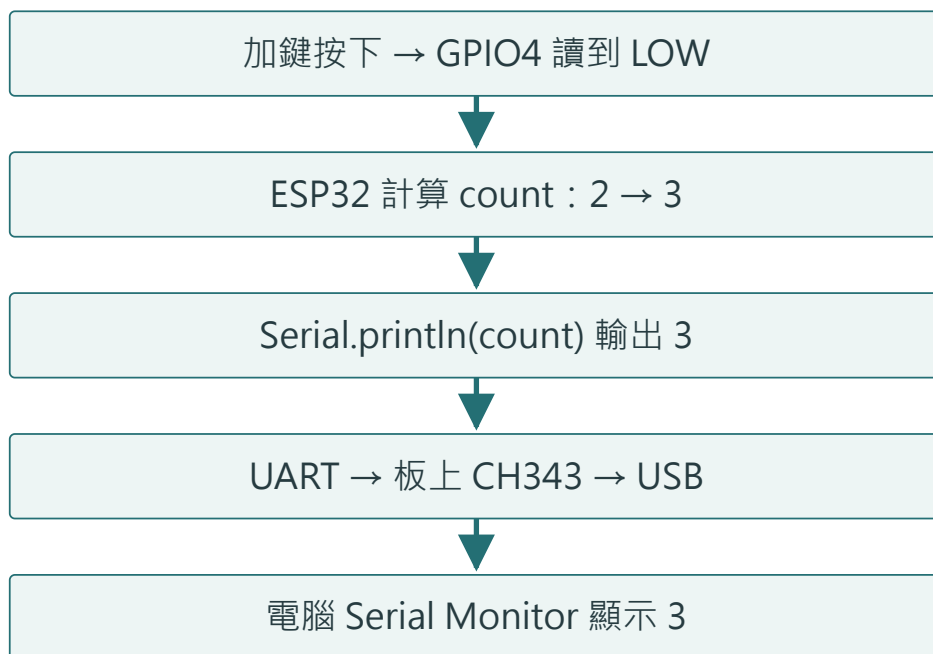
void setup() {
    // 晶片內上拉電阻：放開是 HIGH，按下接 GND 是 LOW。
    pinMode(PLUS_PIN, INPUT_PULLUP);
    pinMode(MINUS_PIN, INPUT_PULLUP);
    Serial.begin(115200);                      // Monitor 也選 115200。
    Serial.println(count);
}

void loop() {
    if (digitalRead(PLUS_PIN) == LOW) {        // 讀到加鍵按下。
        count = count + 1;                    // 計算後存回 count。
        Serial.println(count);                // 只顯示目前數字。
    }
    if (digitalRead(MINUS_PIN) == LOW) {       // 讀到減鍵按下。
        count = count - 1;
        Serial.println(count);
    }
    delay(200);                                // 暫停 0.2 秒，讓輸出容易看。
}
```

程式	意思
count = count + 1;	把目前數字加一，再存回 count。
count = count - 1;	把目前數字減一，再存回 count。
Serial.println(count);	顯示目前數字，然後換行。

數字是誰算的？誰負責顯示？

以 count 原本是 2、這次讀到加鍵按下為例。



資訊流，不是電流方向或新增接線；對應本課板型、COM 接頭與 IDE 設定。

變數是程式記住的一個值

`count = count + 1`；先取出 2，計算 $2 + 1$ ，再把 3 存回 count。= 是存入，不是數學上的相等。

`Serial.println(count)`；把目前的 3 傳給電腦。ESP32 負責計算，Monitor 只負責顯示。

按鈕沒有傳出「3」這個數字；按鈕改變 GPIO 的電壓，程式才決定如何修改 count。

按下哪顆按鈕，哪條路接通？

只有加鍵按下、減鍵放開時，分別看兩條電流路徑。



簡化電路：兩顆上拉電阻在 ESP32 晶片內部，不在按鈕或麵包板內。GPIO 是讀取接點電壓的位置。

加鍵按下：有一條經電阻的回路

3.3V → 晶片內上拉電阻 → 加鍵 → GND。電阻限制電流，GPIO4 與 GND 之間的電壓接近 0V，所以讀到 LOW。不是電流變大就讀 HIGH。

減鍵放開：按鈕處斷開

減鍵這條支路幾乎沒有電流。上拉電阻兩端的電壓差（壓降）很小，GPIO5 對 GND 接近 3.3V，所以讀到 HIGH；不是電壓被「卡住」。

觀察的是什麼	對應內容
電流路徑	供電、上拉電阻、按鈕、GND；不是 count 的傳遞路徑
資訊流	GPIO 讀值 → 程式計數 → Serial → 電腦

兩個 GPIO 都是 INPUT_PULLUP。共用 GND 不代表把 GPIO4 和 GPIO5 短接。不要把 3V3 直接接到 GND；改線前拔 USB。

活動入場人數登記

兩個按鈕登記人員進出，最多容納 20 人。

活動開始時有 0 人。有人進入就按加鍵，有人離開就按減鍵；多人進出時，可以按住按鈕連續登記。

沿用第 28 頁的四條線，不加零件。IDE 選「檔案 → 另存新檔」，把 `counter_practice` 另存成 `room_counter`，再修改程式。

操作	完成後應有的反應
短按加鍵或減鍵	每次加一或減一，顯示目前人數。
按住加鍵不放	人數持續增加，到了 20 就停住，顯示 FULL。
按住減鍵不放	人數持續減少，到了 0 就停住。
兩鍵同時按住	暫停計數，保留目前人數；兩鍵都放開後才能重新操作。
短按板上 RST	重新從 0 開始；放開兩鍵後再登記。

人數只能在 0 ~ 20 之間。長按時以固定、看得清楚的速度連續計數，不能因為程式跑得快就瞬間跳到 20。

放開按鈕、停在上下限或雙鍵暫停時，不要一直印出相同訊息。到 20 顯示 FULL；有人離開後，再顯示更新的人數。

這是人數登記模擬，不會偵測真人。重新上傳前關 Monitor、拔 USB，拍下並拆除板上外接線；只接 COM USB 上傳。完成後拔 USB，照第 28 頁恢復接線，再通電測試。

用這些結果檢查自己的程式

USB 接 COM，Serial Monitor 選 115200。

表中的數字是完成練習後應顯示的人數；每列以「起始人數」為準。

起始人數	操作	預期結果
0	短按加鍵，放開，再短按減鍵	1 → 0
0	按住減鍵	維持 0，不出現負數
18	按住加鍵	19 → 20，顯示 FULL；繼續按仍是 20
2	按住減鍵	1 → 0；繼續按仍是 0
5	兩鍵一起按住	維持 5；只放開一鍵也不變
5，雙鍵暫停中	兩鍵都放開，再短按加鍵	6
任意人數	兩鍵放開，短按 RST	0

回頭觀察 counter_practice

loop() 是會反覆執行的程式區塊。請用第 30 頁的程式與操作結果回答：

1. 從 0 按減鍵，為什麼可能出現負數？要在哪個判斷加入下限？
2. 按住不放，為什麼數字會一直變？「手指按一次」和「loop 執行一次」相同嗎？
3. 兩鍵都讀到 LOW 時，兩個 if 會依序做什麼？如何改成保留人數？
4. 短按 RST 後，數字會保留嗎？哪一行決定起始數字？

請在 room_counter 完成規則，再用表格測試。參考解答另見 week2Ans.pdf。

不啟用上拉，不等於換成導線

只在圖上推演，不修改目前實物接線。

正常的連接 / 處理



只改標記的地方



線表示電路連接；不是實物腳序或完整接線圖。

想一想：保持按鈕放開，只取消 INPUT_PULLUP。GPIO4 還一定是 3.3 V 嗎？如果再按下呢？

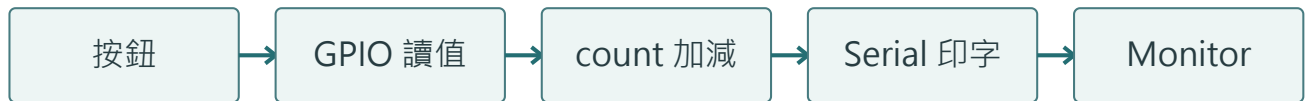
原理提示：取消上拉是斷開通往 3.3 V 的支路，不是用線短接。放開時沒有可靠電壓來源；按下仍接近 GND。

保留已確認的程式與接線。不能用拔帶電線、短接電源或試接未知電壓來驗證反例。

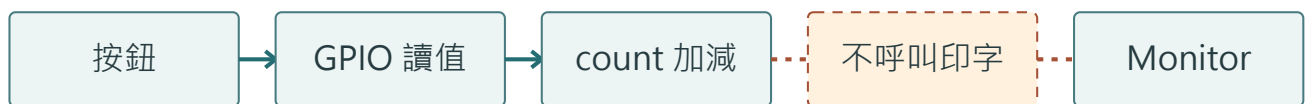
少了印字，按鈕還會改變數字嗎？

只在圖上推演，不修改目前實物接線。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想：只拿掉 `Serial.println(count)`，其他程式與接線保持不變。Monitor 沒有新數字，能判定按鈕壞了嗎？

原理提示：RAM 是程式執行時暫存資料的記憶體。`count` 仍在其中加減；少的是對電腦的回報，不是按鈕的電流路徑。

保留已確認的程式與接線。不能用拔帶電線、短接電源或試接未知電壓來驗證反例。

操作依據與圖片來源

Arduino IDE 操作、ESP32 GPIO 與通斷測試的文件來源。

Arduino 官方文件與截圖

- [Downloading and installing the Arduino IDE 2](#)：安裝總覽，作者 Karl Söderby。
- [How to upload a sketch with the Arduino IDE 2](#)：選單、上傳按鈕，作者 Karl Söderby、Jacob Hylén。
- [Using the Serial Monitor tool](#)：訊息視窗，作者 Karl Söderby。
- [Select board and port in Arduino IDE](#)：板型與連接埠的操作。

Arduino 截圖取自官方 docs-content，未修改影像；原圖採 [CC BY-SA 4.0](#) 授權。舊版截圖只用來辨認入口，不代表本課的板型、速度或版本。

Espressif 官方文件與截圖

- [Getting Started with ESP32 Arduino](#)，Jan Procházka，2025-10-01：偏好設定與管理員畫面，未修改影像；原圖權利屬 Espressif。
- [Installing](#)：穩定版安裝網址。
- [Arduino IDE Tools Menu](#)：設定意義。
- [GPIO](#)：按鈕程式與內部上拉。

實物與量測

板卡、電表與麵包板照片為本課既有實物照片。孔位圖是教學示意；不把圖片當成所有同名商品都相同的保證。通斷原則參考 [Fluke 通斷測試說明](#)。

所有程式輸出示例都是預期格式，不是這次操作的實測紀錄。學習者應以自己實際看到的畫面記錄結果。網路來源查核：2026-09-16。